Lesson plan · for the teacher

Parsons problems

Programming · Level 3 · Lesson 2

Time
40 min

Materials
Printed or handwritten program lines; Scissors, or pre-cut paper strips; One phone to run the answer

Learning goals

- Explain what a Parsons problem is and why it works.
- Design one with good distractors and indentation.
- Run it on paper strips, with no devices.

Activity: Strips on the desk

1. Warm-up (5 min): Give pairs three shuffled lines of a recipe to order. Ask how they knew which came first: each step needed the one before.
2. Teach (5 min): Show a shuffled four-line program on the board and order it together, saying why each line goes where it does, indents included.
3. Solve (15 min): Give each pair a set of strips with one distractor. They arrange the lines and show indentation by shifting strips to the right.
4. Explain (10 min): Pairs explain their order to another pair. Reveal the distractor and ask which misconception it was built on.
5. Wrap-up (5 min): Run the correct program on your phone. Then give the exit ticket.

Prepare strip sets in envelopes and reuse them with every class. Darsi's worksheets print this lesson's Parsons problems ready to cut.

Discussion

- Why must a Parsons problem have exactly one correct order?
- Which distractor worked best, and why?
- When would you move pupils from Parsons to writing code?

Exit ticket

- Write one distractor for a loop that counts to 10, and the misconception behind it.

Try it at school tomorrow
Write a five-line program on paper, cut it into strips and add one distractor. Pairs put it in order and indent it; then check together on the board.

Parsons problems

1 **Two lines in your Parsons problem could go in either order and the program would still work. Is that fair to pupils?**

◯ No: only one order is marked right ◯ Yes: any order is fine

2 **A Parsons problem you could use with a class: swap the first two marks if they are in the wrong order, then print the list. One line doesn't belong.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `temp = marks[0]`

☐ `marks = [14, 9, 17]`

☐ `marks[1] = marks[0]`

☐ `print(marks)`

☐ `if marks[0] > marks[1]:`

☐ `marks[1] = temp`

☐ `marks[0] = marks[1]`

3 **Fill the gaps in one pass of bubble sort: it swaps any neighbours that are in the wrong order.**

```
marks = [14, 9, 17, 11]
for i in _____(len(marks) - 1):
    if marks[i] _____ marks[i + 1]:
        marks[i], marks[i + 1] = marks[i + 1], marks[i]
print(marks)
```

Word bank: `>` `<` `range` `len`

4 **A pupil arranged the strips like this. What does the program print?**

```
total = 0
for m in [4, 6]:
    total = total + m
    print(total)
```

◯ 0 ◯ 4, then 10 ◯ 10

5 **A first Parsons problem for beginners: print the 3 times table from 3 x 1 to 3 x 5, then Done.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `for i in range(1, 6):`

☐ `print("Done")`

☐ `n = 3`

☐ `print(n, "x", i, "=", n * i)`

✎ **Which program from your next lesson would make a good Parsons problem? Which distractor would you add, and which misconception does it test?**

Check your answers

Parsons problems

Programming · Level 3 · Lesson 2

1 **No: only one order is marked right**

A pupil could build a correct program and be marked wrong.

2 **Top to bottom: 3, 1, X, 6, 2, 5, 4**

```
marks = [14, 9, 17]
if marks[0] > marks[1]:
    temp = marks[0]
    marks[0] = marks[1]
    marks[1] = temp
print(marks)
```

A swap needs a temporary place: once marks[0] is overwritten, its old value is gone.

3

```
marks = [14, 9, 17, 11]
for i in range(len(marks) - 1):
    if marks[i] > marks[i + 1]:
        marks[i], marks[i + 1] = marks[i + 1], marks[i]
print(marks)
```

range(len(marks) - 1) visits every neighbour pair; > swaps when the left one is bigger, so big values bubble to the end.

4 **4, then 10**

The print is indented, so it is inside the loop and runs on every turn.

5 **Top to bottom: 2, 4, 1, 3**

```
n = 3
for i in range(1, 6):
    print(n, "x", i, "=", n * i)
print("Done")
```

n must exist before the loop uses it; the times-table line is indented inside the loop; Done is printed once, after it.